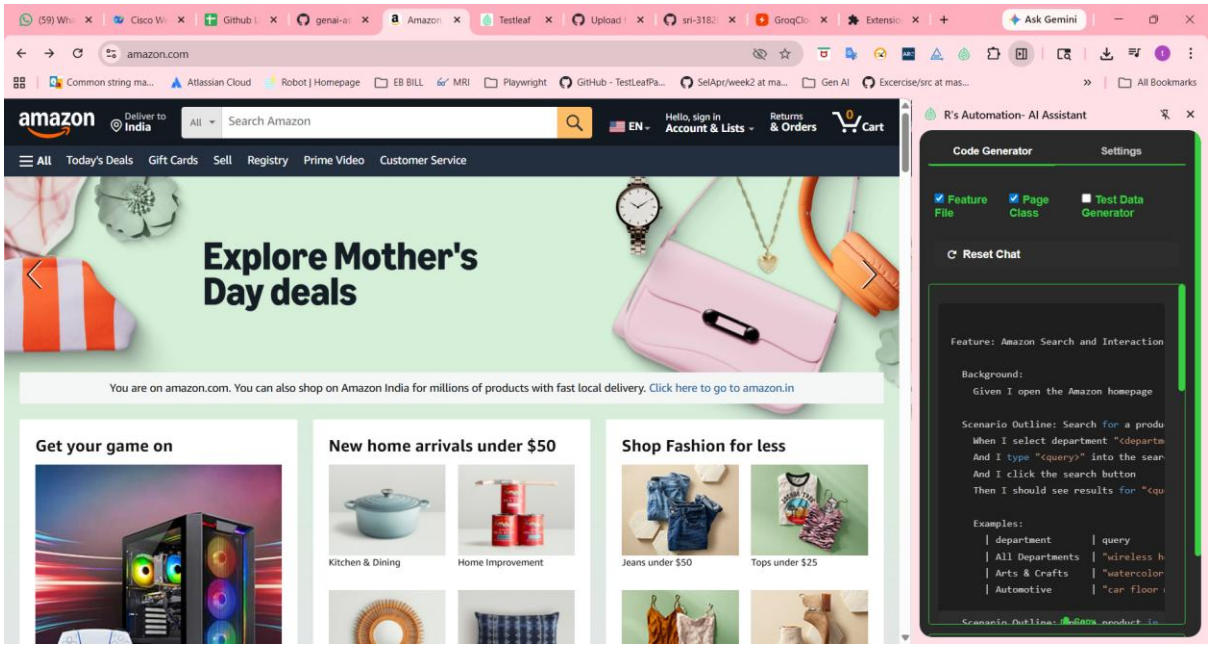
Analyze my entire code base for selenium java (Feature file,Page class,Feature+ Page class)

GO STEP BY STEP .ONCE UP ON MY APPROVAL PROCEED WITH NEXT STEP

--Give me the files in which implemenation is done

--Proceed with Playwright Typescript implementation



Feature: Amazon Search and Interaction Scenarios

Background:

Given I open the Amazon homepage

Scenario Outline: Search for a product in a specific department

When I select department "<department>"

And I type "<query>" into the search box

And I click the search button

Then I should see results for "<query>" in "<department>" category

Examples:

| department | query |

| All Departments | "wireless headphones" |

| Arts & Crafts | "watercolor paper" |

| Automotive | "car floor mats" |

Scenario Outline: Open a product in a new tab and verify details

When I select department "<department>"

And I type "<query>" into the search box

And I click the search button

And I open the first product link in a new tab

Then the product page should display a title containing "<query>"

Examples:

| department | query |

| All Departments | "gaming mouse" |

Scenario Outline: Download the main image of a product

When I select department "<department>"

And I type "<query>" into the search box

And I click the search button

And I open the first product link in a new tab

And I download the product main image

Then the download should complete successfully

Examples:

| department | query |

| All Departments | "bluetooth speaker" |

Scenario Outline: Upload a file when adding a product review

When I select department "<department>"

And I type "<query>" into the search box

And I click the search button

And I open the first product link in a new tab

```
  And I navigate to the "Write a review" section
  And I upload the file "<filePath>"
  Then the file should be attached to the review

  Examples:
    | department       | query              | filePath               |
    | All Departments  | "coffee maker"     | "tests/fixtures/sample.jpg" |

Scenario Outline: Mock search suggestions API and verify UI
  Given I mock the suggestions API to return "<suggestion>"
  When I type "<partialQuery>" into the search box
  Then the suggestion list should display "<suggestion>"

  Examples:
    | partialQuery | suggestion         |
    | "head"       | "headphones"       |


Feature: Amazon Search Functionality

Scenario Outline: Search for a product in a specific department
  Given I open the Amazon homepage
  When I select the department "<department>" from the search dropdown
  And I type "<searchTerm>" into the search box
  And I submit the search
  Then I should see results containing "<searchTerm>"
  And the selected department should be "<department>"

  Examples:
    | department        | searchTerm   |
    | "All Departments" | "Playwright" |
```

```
    | "Arts & Crafts"   | "paint brush"|

import { Given, When, Then, Before, After, setDefaultTimeout } from '@cucumber/cucumber';
import { test as base, expect, BrowserContext, Page } from '@playwright/test';

setDefaultTimeout(60_000);

type World = {
  page: Page;
  context: BrowserContext;
};

const test = base.extend<World>({});

Before(async function (this: World) {
  this.context = await test.browser!.newContext();
  this.page = await this.context.newPage();
});

After(async function (this: World) {
  await this.page.close();
  await this.context.close();
});

/**
 * Helper to retry flaky actions.
 */
async function retry<T>(fn: () => Promise<T>, attempts = 3): Promise<T> {
  let lastError: any;
  for (let i = 0; i < attempts; i++) {
    try {
```

```
      return await fn();
    } catch (e) {
      lastError = e;
      // Small back-off before retrying
      await new Promise(r => setTimeout(r, 500));
    }
  }
  throw lastError;
}

Given('I open the Amazon homepage', async function (this: World) {
  await this.page.goto('https://www.amazon.com/');
  await expect(this.page).toHaveURL(/amazon\.com/);
});

When('I select the department {string} from the search dropdown', async function (this: World,
department: string) {
  const dropdown = this.page.getByLabel('Search in');
  await dropdown.waitFor({ state: 'visible' });
  await retry(() => dropdown.selectOption({ label: department }));
});

When('I type {string} into the search box', async function (this: World, term: string) {
  const searchBox = this.page.getByRole('searchbox', { name: 'Search Amazon' });
  await searchBox.waitFor({ state: 'visible' });
  await searchBox.fill(term);
});

When('I submit the search', async function (this: World) {
  const submitButton = this.page.getByRole('button', { name: 'Go' });
  await Promise.all([
```

```
    this.page.waitForNavigation(),
    submitButton.click(),
  ]);
});

Then('I should see results containing {string}', async function (this: World, term: string) {
  const firstResult = this.page.locator('[data-component-type="s-search-result"]').first();
  await expect(firstResult).toContainText(term, { ignoreCase: true });
});

Then('the selected department should be {string}', async function (this: World, department: string) {
  const dropdown = this.page.getByLabel('Search in');
  const selectedValue = await dropdown.inputValue();
  const selectedOption = dropdown.locator(`option[value="${selectedValue}"]`);
  const selectedText = (await selectedOption.textContent())?.trim();
  expect(selectedText).toBe(department);
});
```